Lab Report: SOAR Containment Routing Rebuild

Oct 25, 2026 · @bot

Objectives and scope

This report documents a self-built, Active Directory-centered SOC home lab used to practice detection engineering and incident response end to end — not just spotting an attack in a SIEM, but automatically triaging and containing it the way a real security operations team would. The lab was built from scratch across five layers: identity and infrastructure (a real domain, `hackme.local`), attack simulation (ten distinct MITRE ATT&CK techniques run with real offensive tooling, not canned samples), log collection (Windows Security auditing, Sysmon, PowerShell logging, all centralized in Splunk), detection engineering (one hand-written Splunk search per technique, each normalized into a common alert schema), and SOAR orchestration (Tines receiving every alert and deciding, automatically, whether and how to contain it, with Wazuh carrying out the containment and Jira tracking the resulting incident).

The objective for this round of work specifically was to fix a structural weakness identified as the lab grew past its first few detections: containment routing logic that was duplicated and hand-edited per alert type did not scale to ten techniques and would not scale further. The goal was to replace that routing with a single, data-driven lookup table, and to prove — with real, live alerts, not simulated test events — that the new routing produced identical, correct containment decisions.

Summary

The lab is a home detection-engineering environment: a Windows Server 2025 domain controller (`hackme.local`), a Windows 10 endpoint (`DESKTOP-01`), and a Kali attack box, feeding Splunk (indexes `win_security`, `sysmon`, `powershell`, `wazuh`) through Sysmon (Olaf's modular config) and Windows event forwarding. Ten MITRE ATT&CK techniques were attacked and detected end to end, with Tines orchestrating automated containment through Wazuh active response.

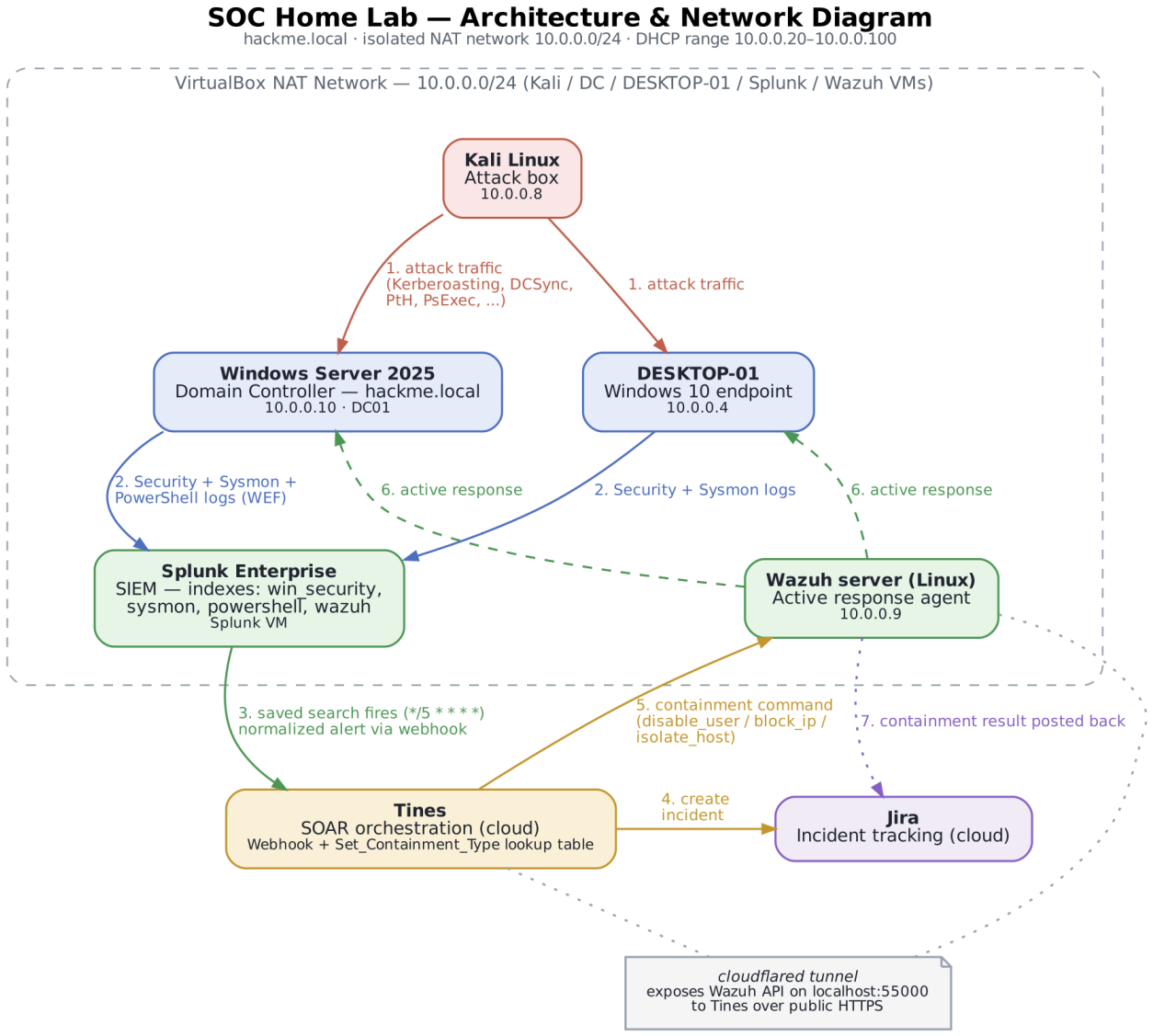
The main engineering change this session was replacing per-alert, hand-maintained regex/condition lists in the Tines routing logic with a single centralized lookup table (`Set_Containment_Type`) that maps each alert's `alert_name` to its containment action.

This removed the need to edit multiple conditions every time a new detection was added, and was verified with three live end-to-end tests: `disable_user`, `block_ip`, and `isolate_host`, all of which routed and executed correctly.

Architecture

Layer	Component	Role
Attack	Kali box	Runs the 10 MITRE technique attacks against the domain
Targets	`hackme.local` DC (Windows Server 2025), `DESKTOP-01` (Windows 10)	Generate Security, Sysmon and PowerShell event logs
Collection	Windows Event Forwarding + Sysmon (Olaf config)	Ships raw events into Splunk's `win_security`, `sysmon`, `powershell` indexes
Detection	Splunk saved searches (scheduled, `*/5 * * * *`)	One search per technique; each emits a normalized alert (`alert_name`, `mitre_id`, `severity`, `user`, `host`, `src_ip`, `event_time`) to a Tines webhook
Orchestration	Tines	Receives the webhook, classifies the alert, decides public vs. private IP, routes to the correct containment action
Containment	Wazuh active response, run on the DC/endpoint	Executes `disable_user`, `block_ip`, or `isolate_host` depending on the alert type

Each detection follows the same pipeline: Splunk search fires on a schedule → posts a normalized JSON payload to the Tines webhook → Tines classifies the alert, enriches the source IP, de-duplicates against open Jira tickets and decides the containment action → a human approves where required → Wazuh carries out the response and the result is posted back to the Jira ticket. The ten techniques are Kerberoasting (T1558.003), AS-REP Roasting (T1558.004), DCSync (T1003.006), Credential Dumping via Remote Registry (T1003.002), Pass-the-Hash (T1550.002), Encoded PowerShell Execution (T1059.001), Scheduled Task Persistence (T1053.005), Password Spraying (T1110.003), Rogue Privileged Account Creation (T1136.002) and Lateral Movement via PsExec (T1021.002).



Detection inventory

Each of the ten techniques below was executed with real offensive tooling (primarily Impacket, run from the Kali box against the domain), detected by a dedicated Splunk saved search, and routed through the same Tines/Wazuh containment pipeline described above. `containment_type` is the value each alert now resolves to through the `Set_Containment_Type` lookup table.

#	Splunk saved search	MITRE ID	Primary data source	`containment_type`
1	SOC - Kerberoasting	T1558.003	`win_security` EventCode 4769 (service ticket requests, encryption type)	`disable_user`
2	SOC - AS-REP Roasting	T1558.004	`win_security` EventCode 4768 (pre-auth not required)	`disable_user`
3	SOC - DCSync	T1003.006	`win_security` EventCode 4662 (directory replication rights used)	`disable_user`
4	SOC - Credential Dumping via Remote Registry	T1003.002	`win_security` EventCode 5145 (remote SAM/SYSTEM hive access over SMB)	`isolate_host`
5	SOC - Pass-the-Hash	T1550.002	`win_security` EventCode 4624 (NTLM network logon)	`isolate_host`
6	SOC - Encoded PowerShell Execution	T1059.001	`powershell` EventCode 4104 (Script Block Logging)	`isolate_host`
7	SOC - Scheduled Task Persistence	T1053.005	`win_security` EventCode 4698 (task created)	`isolate_host`
8	SOC - Password Spraying	T1110.003	`win_security` EventCode 4771 (Kerberos pre-auth failures across many accounts)	`block_ip`
9	SOC - Rogue Privileged Account Creation	T1136.002	`win_security` EventCode 4720 (user created) + 4728 (added to privileged group)	`disable_user`

#	Splunk saved search	MITRE ID	Primary data source	`containment_type`
10	SOC - Lateral Movement via PsExec	T1021.002	`win_security` EventCode 7045 (service installation)	`isolate_host`

All ten searches run on a five-minute schedule (`*/5 * * * *`) and post a normalized JSON payload to the same Tines webhook: `alert_name` , `mitre_id` , `severity` , `user` , `host` , `src_ip` , and `event_time` . Keeping that schema identical across all ten detections is what makes the single lookup table possible — `Set_Containment_Type` only ever has to read one field, `alert_name` , to decide the response for any of them.

Routing rebuild: from regex lists to a lookup table

Before: each containment path (`disable_user` , `block_ip` , `isolate_host`) had its own Tines condition node matching `alert_name` against a hand-maintained regex, e.g. extending `Bypass_For_PTH` 's pattern with `^(Pass-the-Hash|LSASS/SAM Credential Dumping|...|Suspicious Service Installation \(Lateral Movement\))$` . Every new detection meant editing multiple condition nodes across the story, and it was easy to add an alert to one list and forget another.

After: a single formula/Set Context node, `Set_Containment_Type` , holds one local lookup table, `LOCAL.containment_table` , mapping each `alert_name` to its `containment_type` :

```
containment_type = FIND(LOCAL.containment_table, LAMBDA(r, r.alert =
webhook.body.result.alert_name)).type
```

Unlisted alerts default to `disable_user` . The node also resolves `should_contain` and `protected_account` on the same event, so downstream logic reads one clean record instead of re-deriving it. The flow simplified to:

```
Webhook → Set_Containment_Type → Should_Contain → Requires_Approval →
(disable_user | block_ip | isolate_host) → Jira_Containment_Result
```

Adding a new detection to containment now means adding one row to the lookup table, not touching every branch condition. The table was built and verified in isolation first (wired in parallel off `Create_Jira_Incident`) before being cut into the main sequence, to confirm the formula logic before touching approval/dedup wiring.

Safety controls and operational hardening

The final story goes beyond routing. It adds controls that keep automated containment from doing damage or creating noise:

- **Human approval.** Every `disable_user` and `isolate_host` action waits for an analyst to move the Jira ticket to *Containment Approved* or *Containment Rejected* (`Request_Approval`, `Wait_For_Approval`, `Get_Jira_Approval_Status`). Tines re-checks every 60 seconds; with no decision after 4 hours the story takes no action and notes the timeout on the ticket. `block_ip` is the one action that skips approval, because blocking an external spraying source is low-risk and time-sensitive.
- **Protected accounts.** `Set_Containment_Type` also holds a `protected_accounts` list (`administrator`, `krbtgt`, `guest`, `svc_sql`, `svc_lab`). The `should_contain` flag stops account-level containment for these identities, while host isolation still proceeds because it acts on the machine. The Wazuh scripts add a second, host-side guard (never disable machine accounts, `krbtgt` or `Administrator`; never block the DC, the endpoint or loopback).
- **Duplicate suppression.** `Build_Dedup_Key` builds a key from alert, user and host and `Search_Open_Duplicate` checks Jira. A repeat alert comments on the existing open ticket instead of creating a new one.
- **Dynamic agent lookup.** `Lookup_Wazuh_Agent` resolves the Wazuh agent ID from the alert's host name before `isolate_host` runs, and files a Jira note if the agent cannot be found.
- **Failure visibility.** A Wazuh authentication failure, a missing agent, a rejection and an approval timeout each write back to the Jira ticket instead of failing silently.

Testing and results

Three live end-to-end tests confirmed the rebuilt routing against the real pipeline, each injected as a genuine alert through the webhook rather than a synthetic Tines test event:

Test	Expected `containment_type`	Result
`disable_user`	`disable_user`	Correctly classified and executed by Wazuh active response
`block_ip`	`block_ip`	Correctly classified and executed by Wazuh active response
`isolate_host`	`isolate_host`	Correctly classified and executed by Wazuh active response

All three resolved through `Set_Containment_Type` to the expected `containment_type`, passed `Should_Contain` / `Requires_Approval`, and landed in the correct containment branch, each still reporting its result back through its own `Jira_Containment_Result` node. One related finding confirmed during this round of testing: `wmiexec`-based attacks (e.g. attack #9, Rogue Admin Account Creation) reliably also trigger a correlated Pass-the-Hash alert, since `wmiexec` authenticates via NTLM hash before executing any command. This is expected multi-technique detection, not a false positive or routing bug, and is worth documenting once as a general pattern rather than flagging per attack.

Issues, lessons learned, and next steps

Issues hit along the way

- Early cleanup scripts re-ran `Enable-ADAccount -Identity lab_dip` reflexively after every attack, even when that attack never touched `lab_dip`; worth checking per-attack whether a cleanup step is actually needed rather than copying the template forward.
- Several detections needed small correctness fixes after real log review (field name mismatches, AES vs. RC4 encryption type assumptions, blank `Account_Name` on service-install events) — synthetic test data would have missed these.
- The Kerberoasting detection had to be tuned against the real `msDS-SupportedEncryptionTypes` value on `svc_sql`, since the lab's effective encryption type didn't match assumptions until it was set explicitly.

Lessons learned

- A single lookup table scales far better than per-branch regex conditions once past 3–4 detections — this is the pattern to default to for any future routing logic (containment type, severity, team assignment).

- Validate new Tines logic in isolation (wired in parallel, off to the side) before cutting it into the live sequence, so a bug in the new logic can't break the existing pipeline mid-test.
- `wmiexec`-based techniques will always co-trigger Pass-the-Hash; that correlation is a detection feature, not noise, and should be documented so it isn't mistaken for a bug in future attacks.

Suggested next steps

- Add the remaining attack techniques' alert names to `LOCAL.containment_table` as they're built, instead of writing new condition nodes.
- Correlate PsExec/service-install alerts with a source IP (4624 logon source or Sysmon EID 3) to close the current `src_ip_available` gap noted during testing.
- Consider moving `containment_table` from a Tines LOCAL variable to a shared/global table or external lookup, so it can be reused across stories and edited without redeploying the formula node.
- Write a short runbook per containment action (`disable_user`, `block_ip`, `isolate_host`) covering rollback/cleanup steps, since several attacks required manual cleanup (deleting test accounts, removing stray services).

Conclusion

The lab now demonstrates a complete, working detection-and-response loop across ten distinct attack techniques, built and tested on real infrastructure rather than a tutorial dataset: real attacks, real log data, hand-written detections, and automated containment that was proven correct with live alerts rather than synthetic test events. The specific improvement made in this round of work — collapsing per-alert regex routing conditions into a single `Set_Containment_Type` lookup table — is a small change in line count but a meaningful one in engineering maturity: it moves the lab from a pipeline that had to be edited in multiple places for every new detection to one that scales by adding a single row to a table. That pattern (data-driven routing over hardcoded conditionals) is now the template to apply anywhere else in the pipeline that currently branches on alert type.

With the routing layer solid, the highest-value next steps are the ones listed above: closing the source-IP gap on host-based alerts, moving the containment table out of a local Tines variable so it can be maintained independently of the formula node, and writing short rollback runbooks for each containment action so cleanup after a test run is as repeatable as the detection itself.